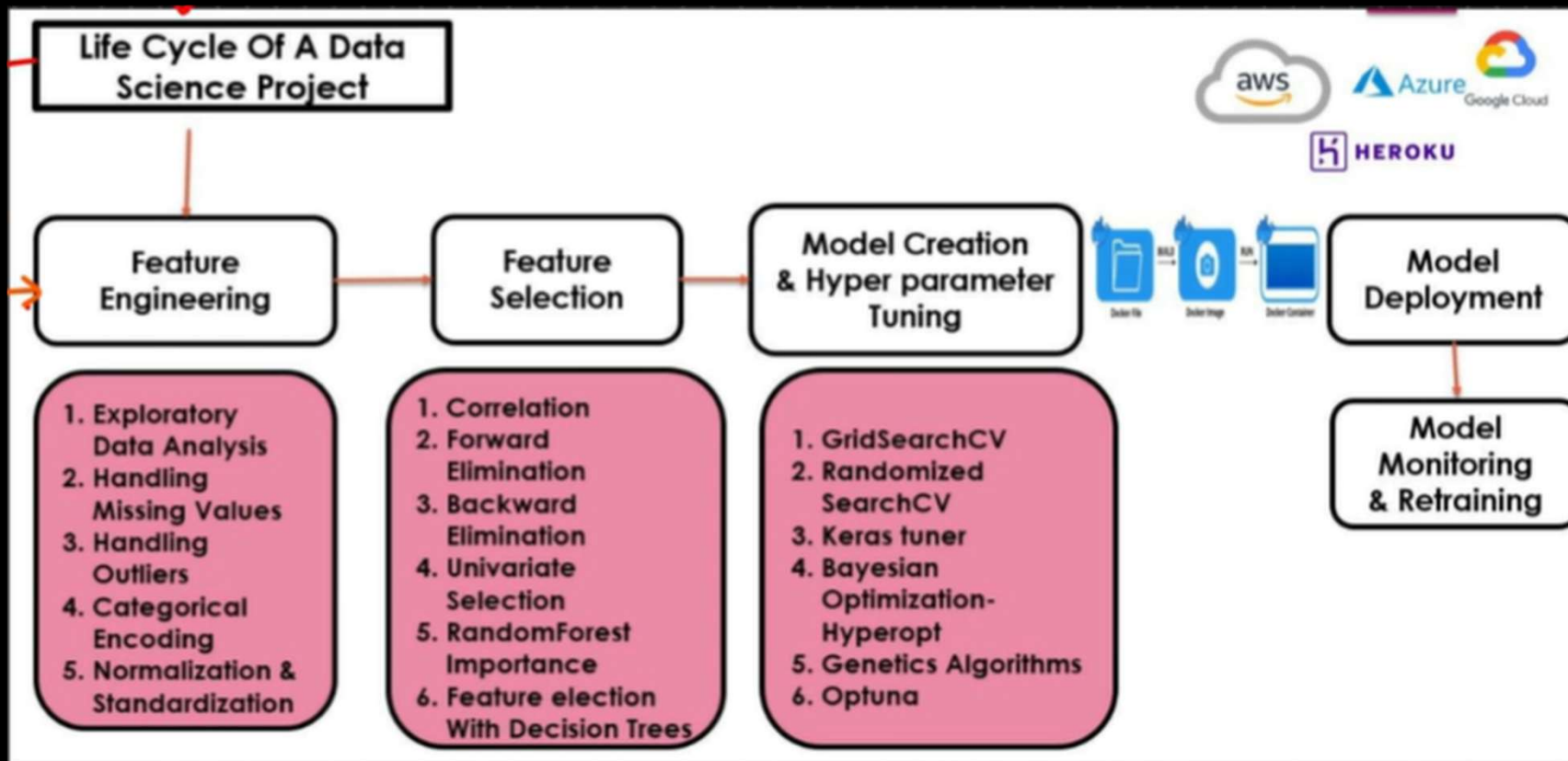




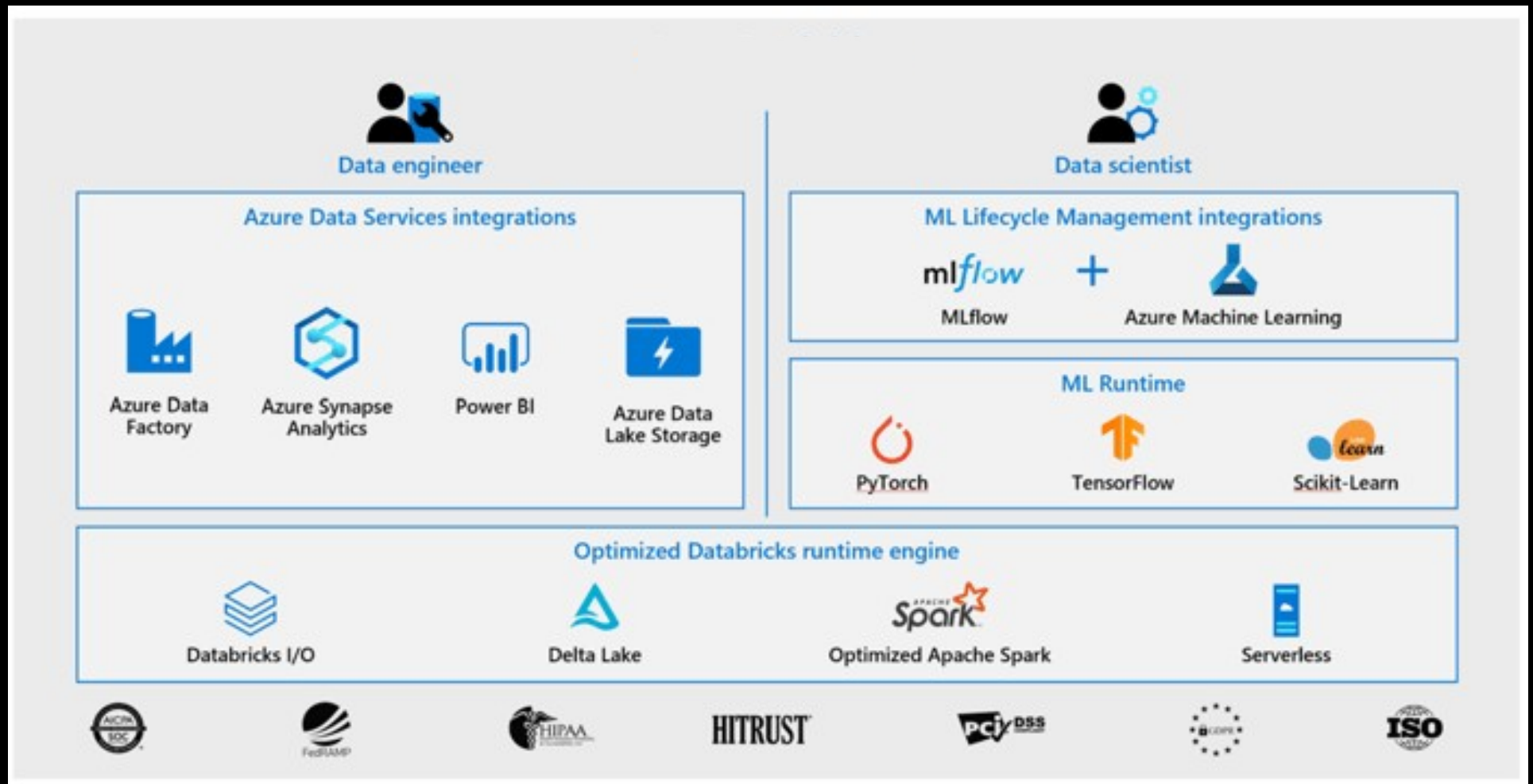
Generative AI Academy

ML Ops

Data Science Project Steps



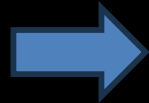
MLOps with Azure



Enterprise Machine Learning with Microsoft Azure

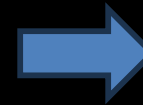
Data ingestion

- **Azure Data Factory**
Low code/no code tool:
Visual pipeline designer, control flows, event triggers, scheduling
- Data ingestion, ETL at scale
- Integrates with Databricks, Synapse
- For async http calls, Python asyncio, aiohttp are suitable



Data Eng., model training

- **Azure Databricks**
data cleaning, feature engineering at scale with Apache Spark
- Distributed Model training with Apache Spark. Integrates with Hadoop
- Notebooks
- Metadata, Delta Tables



Data Storage

- Parquet files in Azure Data Lake or AWS S3
- Azure Synapse
If you require complex sql queries, reporting or ad-hoc analysis on structured data

Databricks

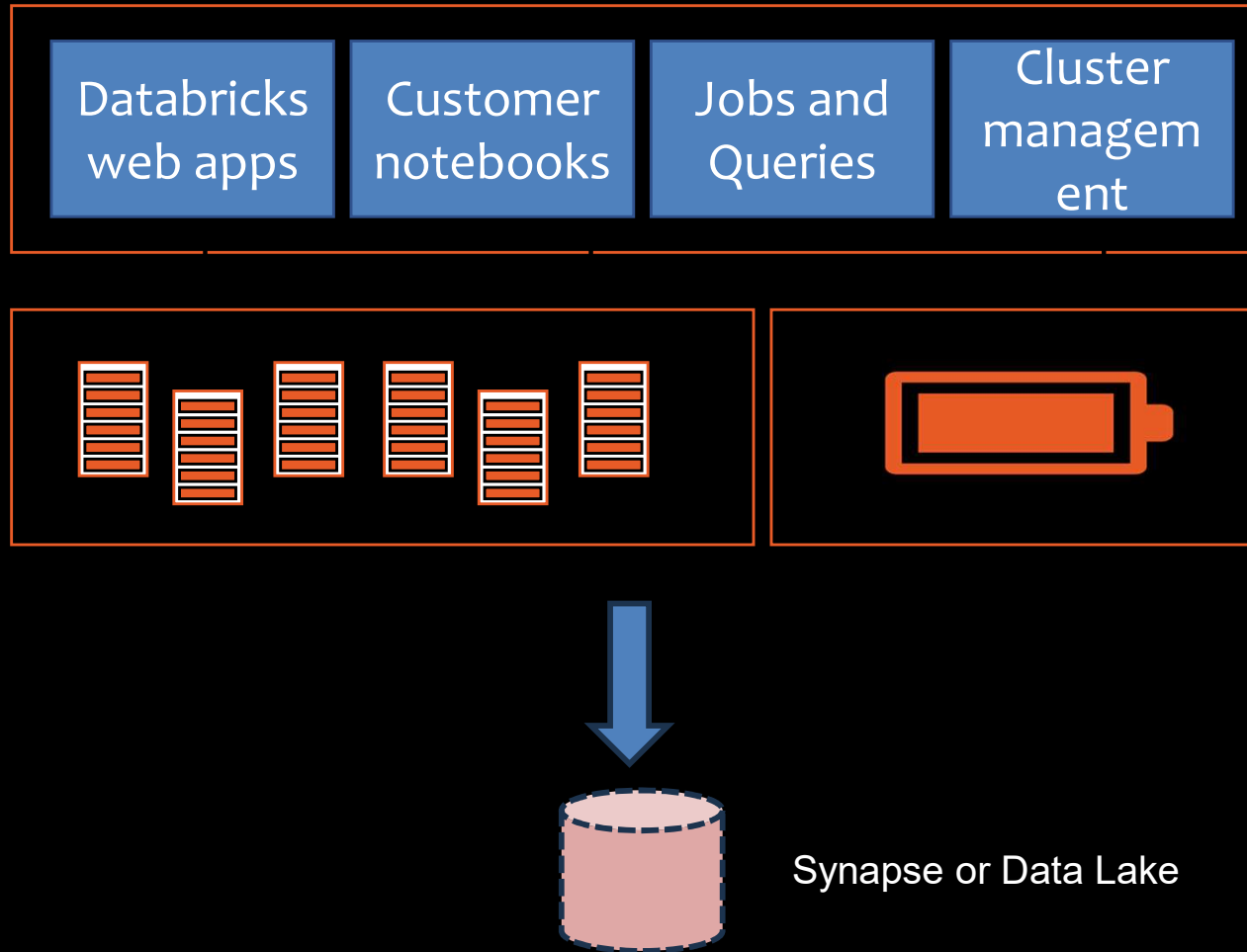
Databricks

An enterprise software company founded by the creators of Apache Spark. The company has also created Delta Lake, MLflow, and Koalas, open source projects that span data engineering, data science, and machine learning.

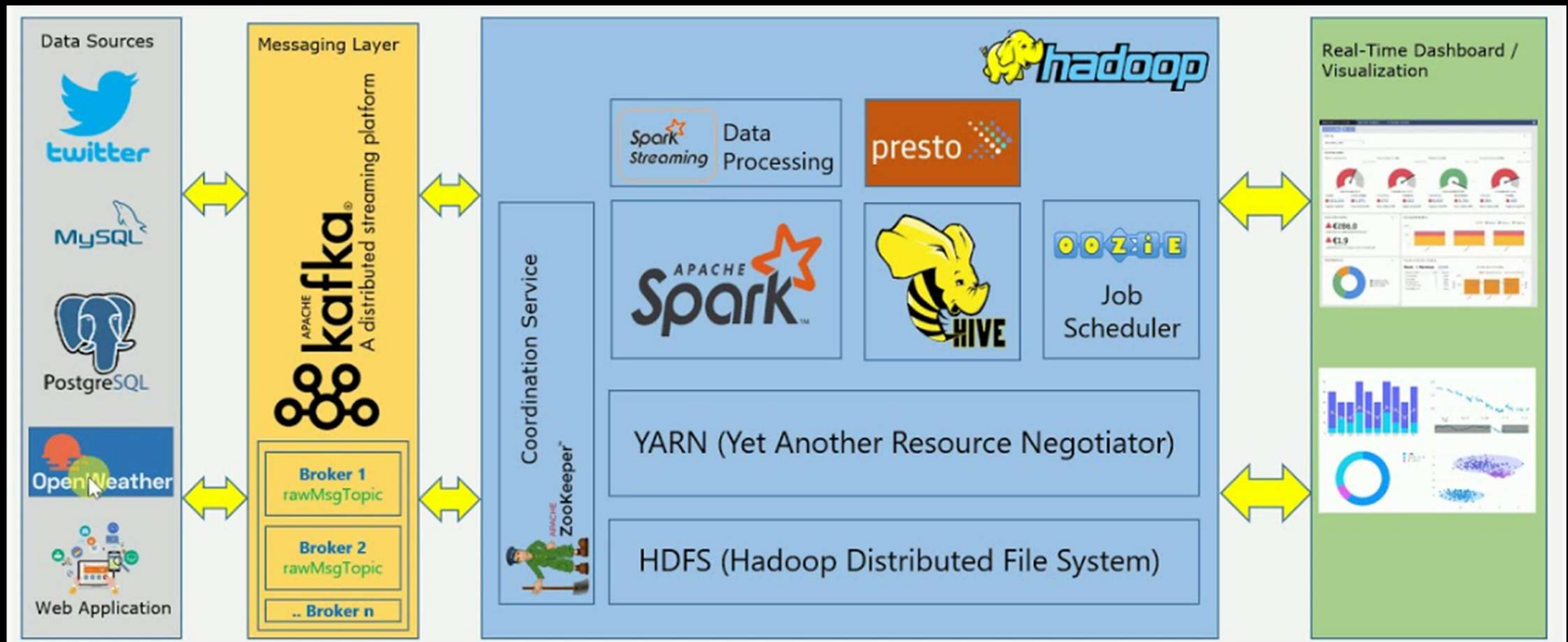


- Manages Spark cluster
- Integrates with Kubernetes, Hadoop

Databricks

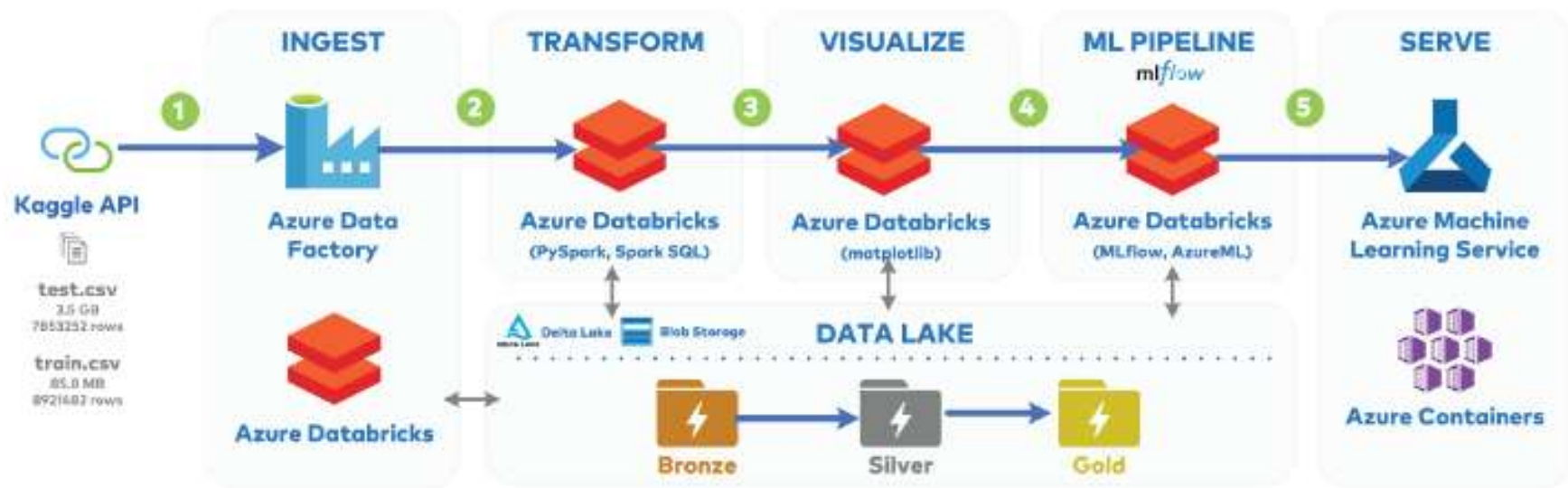


Data processing – Enterprise architecture



MLOps with Azure

Understanding MLOps with Azure Databricks

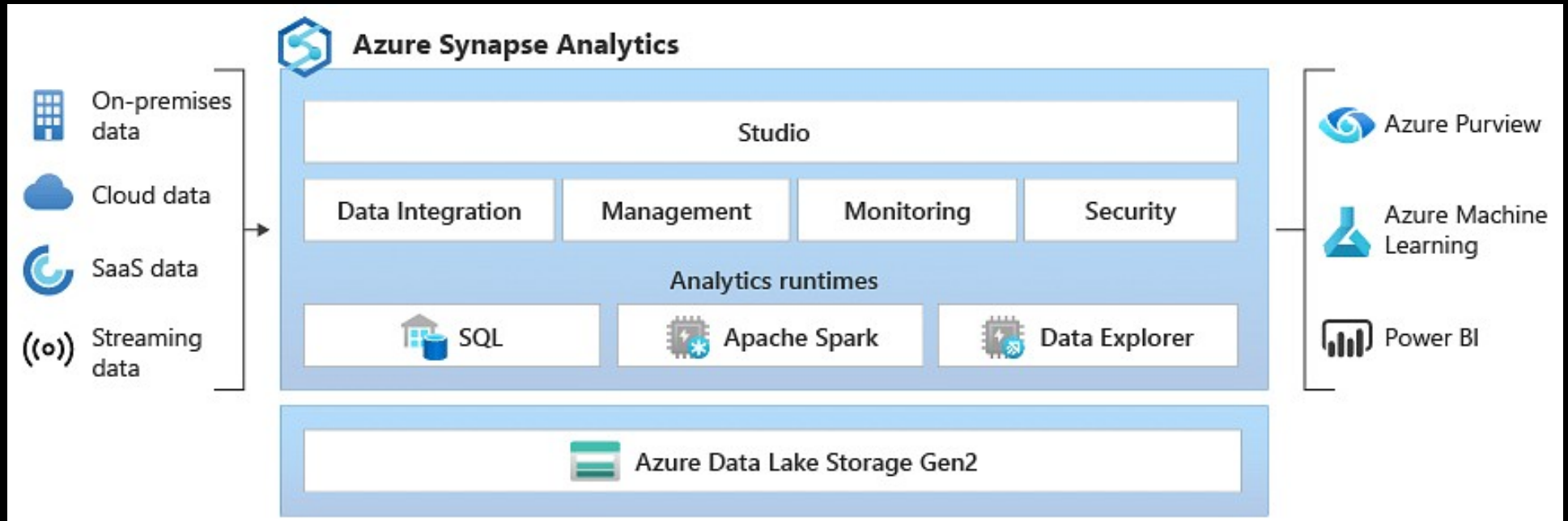


slalom

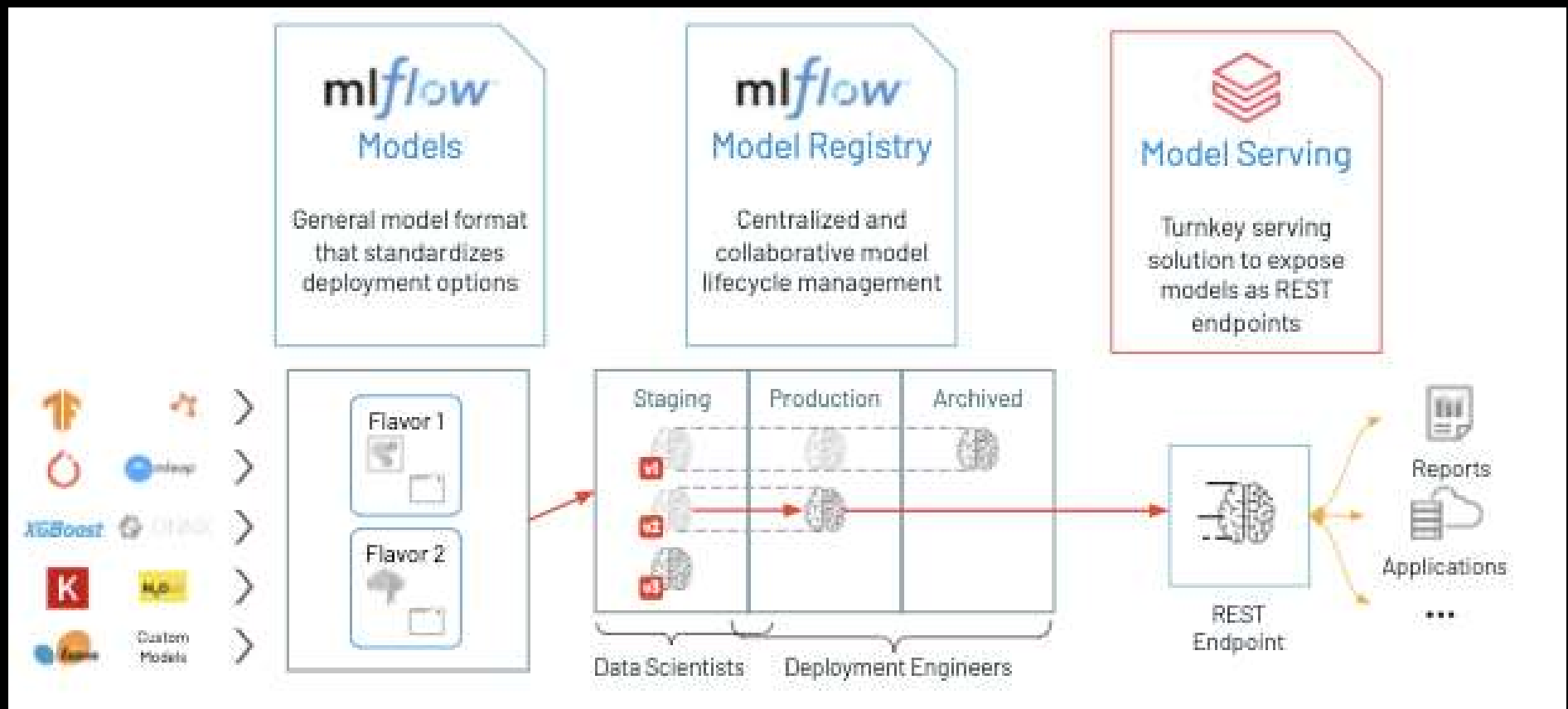
databricks

Microsoft

Azure Synapse



MLflow and Databricks



MLflow Models



ML
Frameworks



Standard for ML
models



Inference Code



Batch & Stream
Scoring



Serving Tools

In what format can the model be saved?

Machine Learning models

Format	Best For	File Extension	Framework Support	Cross-Platform	Speed	Security
Pickle	Standard Python usage	.pkl	Scikit-learn only	No	Fast	Low
Joblib	Large models, pipelines	.joblib	Scikit-learn only	No	Faster	Low
ONNX	Cross-framework usage	.onnx	PyTorch, TensorFlow	Yes	Fast	Moderate
PMML	Enterprise applications	.pmml	Business platforms	Yes	Moderate	High
MLflow	MLOps and cloud deployment	MLflow Directory	Cloud platforms	Yes	Moderate	High

In what format can the model be saved?

Deep Learning models

Format	Framework Support	File Extension	Model Type	Best For	Pros	Cons
PyTorch	PyTorch, Hugging Face Transformers	.pth, .pt	Transformers, GANs	Training, fine-tuning, inference	Easy to save and load, widely used in research	Framework-specific, not cross-platform
TensorFlow SavedModel	TensorFlow, Keras	SavedModel directory	Transformers, GANs	Production, deployment, training	Full model serialization, cross-platform	Complex structure, large size
HDF5 (Keras)	TensorFlow, Keras	.h5	Any deep learning model	Sharing and storage of Keras models	Compact, easy to share	Keras-specific, not as efficient for large models
ONNX	PyTorch, TensorFlow, Scikit-learn	.onnx	Any deep learning model	Cross-framework deployment	Platform-independent, optimized for inference	Conversion may lose precision
Safetensors	Hugging Face Transformers	.safetensors	Transformers, LLMs	Secure and efficient storage	Memory-safe, faster loading	Limited support outside Hugging Face ecosystem
MLflow	MLflow, Any ML Framework	Directory structure	Any ML model	Experiment tracking and deployment	Built-in versioning and deployment tools	Requires MLflow infrastructure
TensorRT	NVIDIA GPUs	.engine	Any deep learning model	High-performance inference on GPUs	Significant speedup for NVIDIA hardware	Hardware-specific, complex setup

Hands-On lab

- saving the model as pickle file
- loading the saved model and doing inferencing

Next Steps

Deploying the model

- Docker container
- MLFlow Service

Model Serving with REST Endpoint

- Azure ML Studio
No code deployment of ML model as REST service via Azure App
- MLFlow
- FastAPI
- Flask

Deploying the models as API

Flask API

- More traditional, explicit approach
- Manual input validation
- Uses JSON for request/response schemas
- Runs on port 5000 by default
- Uses Waitress as the production server

FastAPI API

- Modern, automatic OpenAPI documentation
- Automatic input validation via Pydantic models
- Type hints and data validation built-in
- Runs on port 8000 by default
- Uses Uvicorn as the ASGI server
- Better performance due to async support

Deploying the models as API

Method	Best for	Performance	Ease of Use	Scalability
Flask	Small-scale, simple APIs	Medium	Easy	Low
FastAPI	High-concurrency APIs	High	Moderate	High
MLflow	MLOps, production tracking	High	Complex	Very High

Production-ready servers

- Waitress for Flask
- Uvicorn for FastAPI

Hands-On lab with Fast API and Flask

Containerizing the model with Docker

Containerizing a model helps with the following:

- **Consistent environment**
requirements.txt with exact versions of python packages is baked into the docker image, so that users of the model do not have to worry about it
- **Simplified deployment**
A Docker image can be easily shared and deployed on any platform supporting Docker
- Efficient resource utilization, version control, reproducibility, helps with devops, easier scalability, monitoring etc.
(standard container benefits)

Demo of docker containerization of the model